# Web Development Foundations

Week 8 – "Async Time Matters"

*Day 2: "Async Timing – Requests, Promises, & Loading"*

1

## Reconnect

*Monday's key question*:

- What happens now?
- What happens later?

***Today we apply that same question to***:

- loading
- requests
- future results
- JSON-shaped data

2

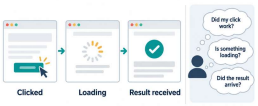
## Timing Is A User Experience Issue

Async is not only a code issue.

It affects the user:

- Did my click work?
- Is something loading?
- Did the result arrive?
- Should I wait or try again?

***"Good pages explain waiting!"***




3

## What Counts as Success Today

- trace what runs now and later
- explain why data is not instant
- describe `fetch` as starting a request
- describe a Promise as a future result
- read simple JSON structure before using it

4

## From Delay To Request




Key Analogy:

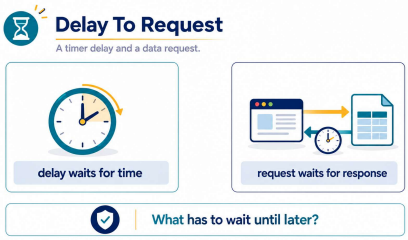
*A delay waits for time.*

*A request waits for a response.*

Both create the same beginner question:

- What can happen now?
- What has to wait until later?

5




## Delay To Request

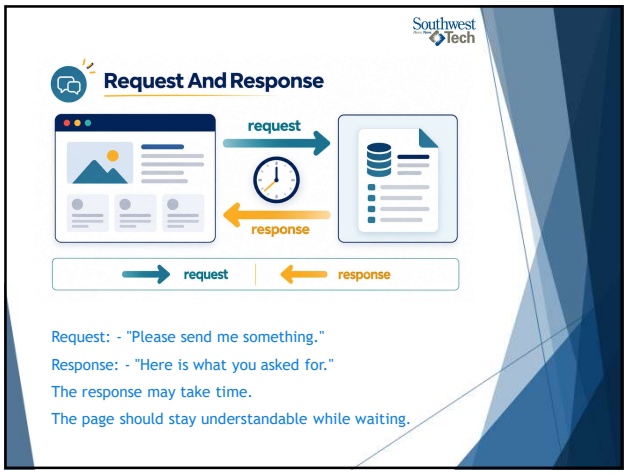
6


Southwest Tech
What We Will Use Today
request
response
`fetch()`
Promise
`async`
`await`
JSON
loading feedback

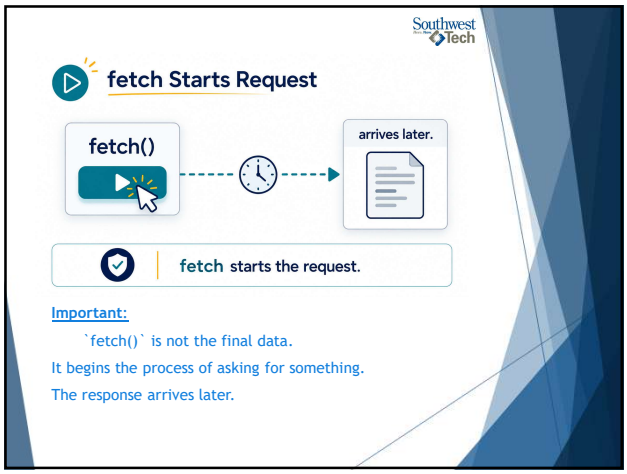
7


Southwest Tech
Request Toolbox
Request-shaped async concepts.
request
response
fetch()
Promise
async
await
JSON
loading feedback
Request-shaped async concepts.
Today's Toolbox

8


Southwest Tech
Request And Response
request
response
request
response
Request: - "Please send me something."
Response: - "Here is what you asked for."
The response may take time.
The page should stay understandable while waiting.

9



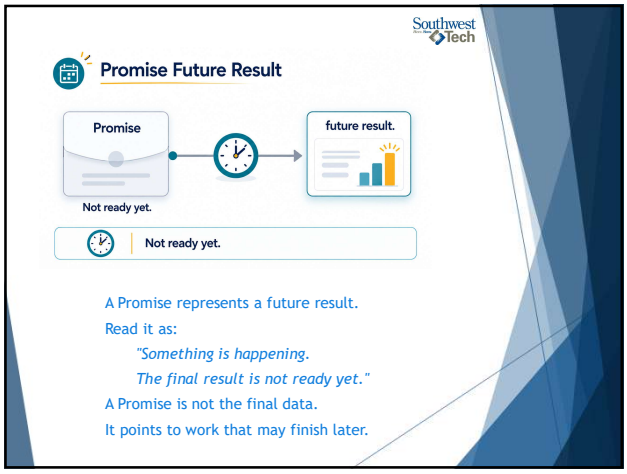

Important:

`fetch()` is not the final data.

It begins the process of asking for something.

The response arrives later.

10




A Promise represents a future result.

Read it as:

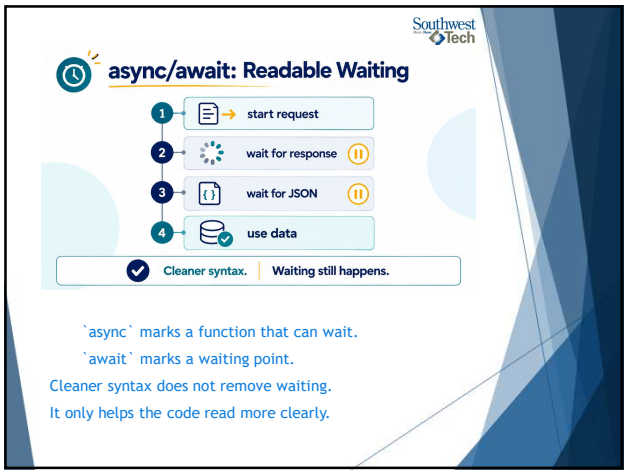
*"Something is happening.*

*The final result is not ready yet."*

A Promise is not the final data.

It points to work that may finish later.

11



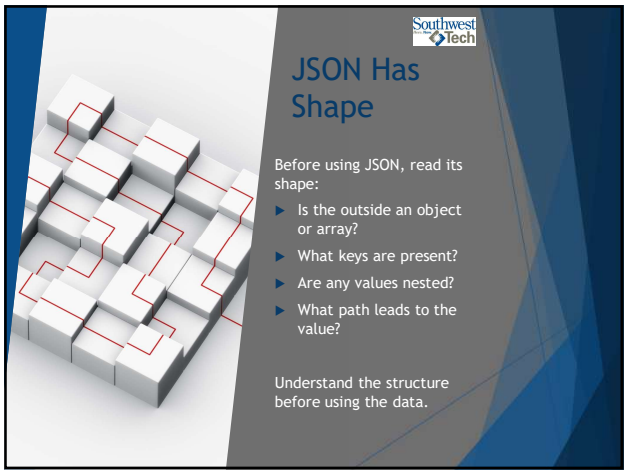

`async` marks a function that can wait.

`await` marks a waiting point.

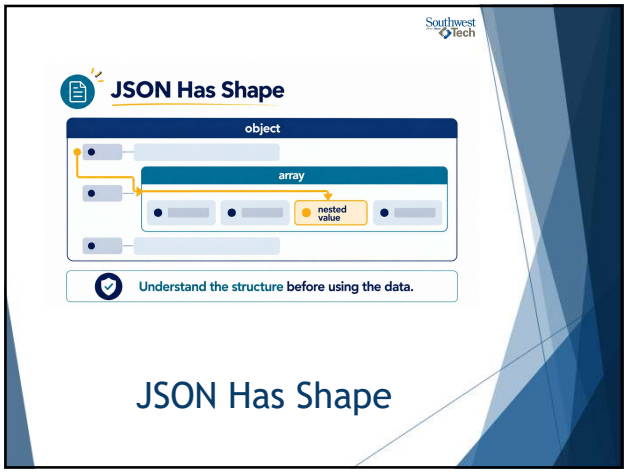
Cleaner syntax does not remove waiting.

It only helps the code read more clearly.

12


Southwest Tech
JSON Has Shape
Before using JSON, read its shape:
Is the outside an object or array?
What keys are present?
Are any values nested?
What path leads to the value?
Understand the structure before using the data.

13


Southwest Tech
JSON Has Shape
object
array
nested value
Understand the structure before using the data.
JSON Has Shape

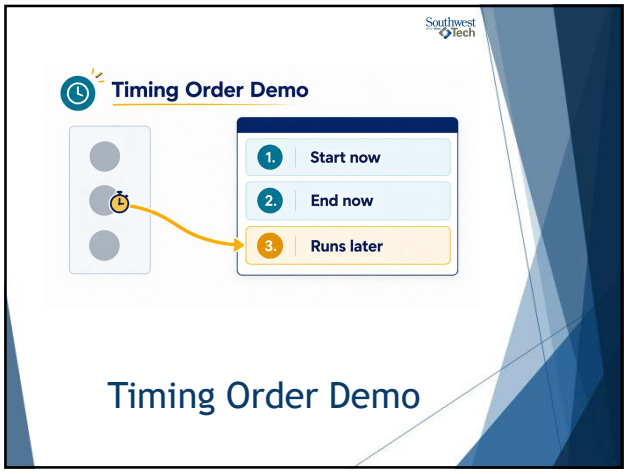
14


Demo
"Timing Order"

15


Southwest Tech
What to Watch For
start now
schedule later work
finish current work now
later work appears after the delay
Typed order and finished order are not always the same.
Q2
Q3

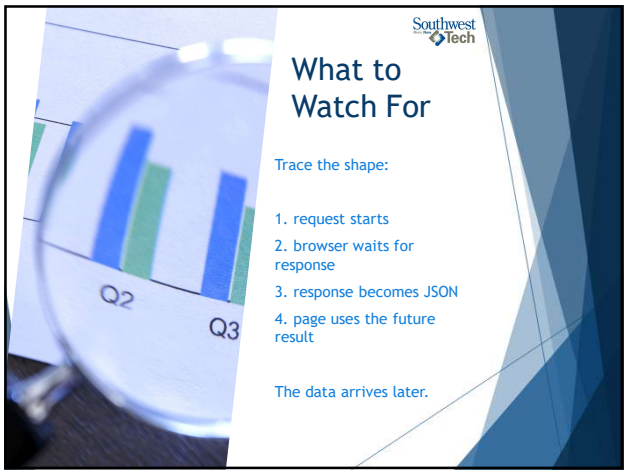
16


Southwest Tech
Timing Order Demo
1. Start now
2. End now
3. Runs later
Timing Order Demo

17


Demo
"Fetch Timing"

18




## What to Watch For

Trace the shape:

1. request starts
2. browser waits for response
3. response becomes JSON
4. page uses the future result

The data arrives later.

19

## Fetch Timing Demo




Fetch Timing Demo

20

## Waiting Still Happens



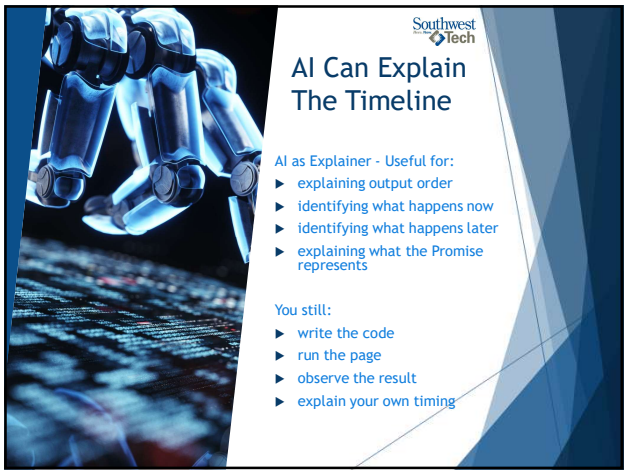

Do not let clean syntax trick you.

This code may look step-by-step:

- fetch
- await response
- await JSON
- use data

But the request still takes time.

21



## AI Can Explain The Timeline

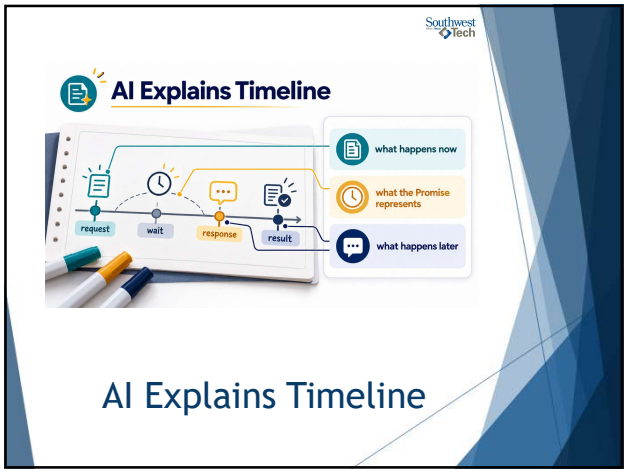
AI as Explainer - Useful for:

- explaining output order
- identifying what happens now
- identifying what happens later
- explaining what the Promise represents

You still:

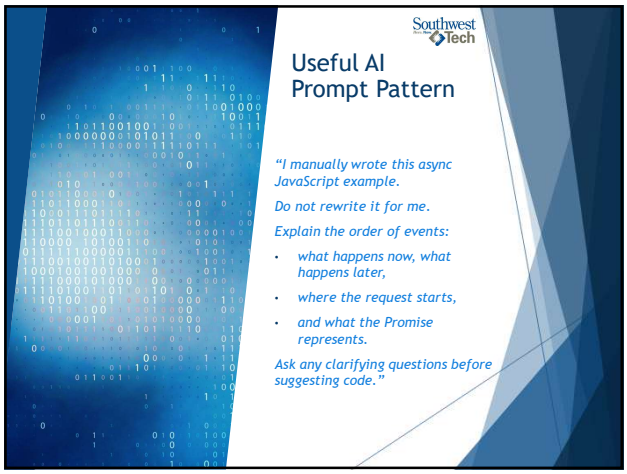
- write the code
- run the page
- observe the result
- explain your own timing

22




## AI Explains Timeline

23



## Useful AI Prompt Pattern

*"I manually wrote this async JavaScript example.*

*Do not rewrite it for me.*

*Explain the order of events:*

- *what happens now, what happens later,*
- *where the request starts,*
- *and what the Promise represents.*

*Ask any clarifying questions before suggesting code."*

24

## Useful AI Prompt Pattern

| | |
|---|---|
| **Context:** | I manually wrote this async JavaScript example. |
| **Constraint:** | Do not rewrite it for me. |
| **Timeline:** | what happens now and what happens later. |
| **Request:** | where the request starts. |
| **Promise:** | what the Promise represents. |

Ask one question before suggesting code.

# Useful AI Prompt Pattern

25



## Lab Refinement

In Lab 08:

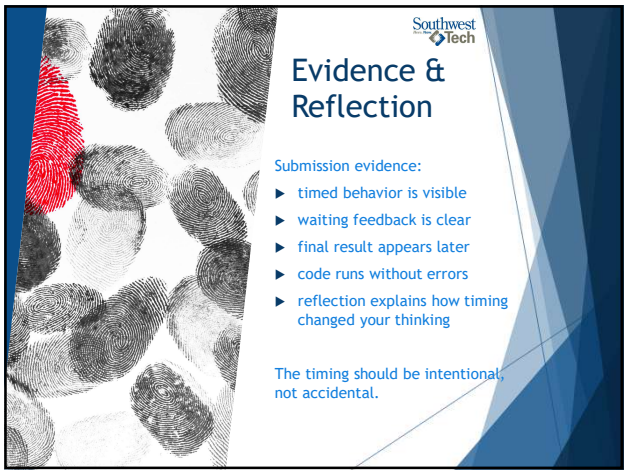
Improve your async behavior.

Choose at least one, (if you are ahead - do all of them):

- add a second timed interaction
- improve the waiting message
- make the sequence clearer
- structure the timed code better

Keep it explainable.

26



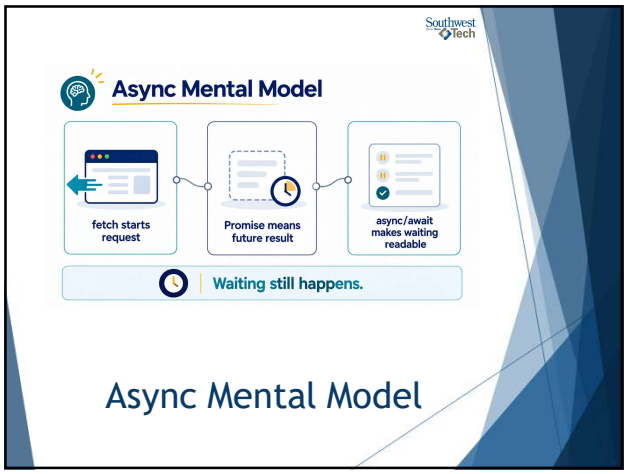
## Evidence & Reflection

Submission evidence:

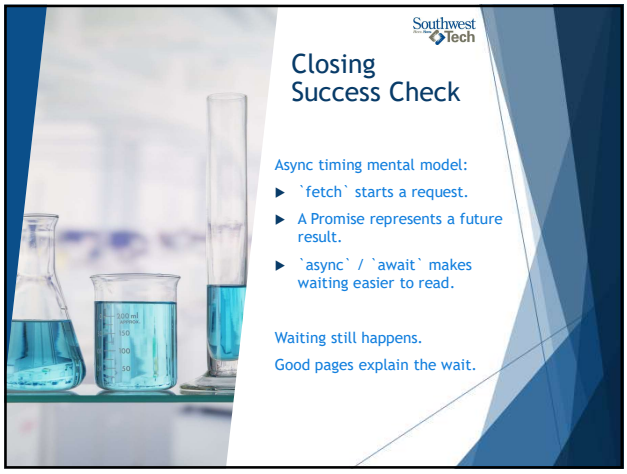
- timed behavior is visible
- waiting feedback is clear
- final result appears later
- code runs without errors
- reflection explains how timing changed your thinking

The timing should be intentional, not accidental.

27


Southwest Tech
Async Mental Model
fetch starts request
Promise means future result
async/await makes waiting readable
Waiting still happens.
Async Mental Model

28


Southwest Tech
Closing Success Check
Async timing mental model:
`fetch` starts a request.
A Promise represents a future result.
`async` / `await` makes waiting easier to read.
Waiting still happens.
Good pages explain the wait.

29


Southwest Tech
Thank you!
Have Fun!

30